Guía de desarrollo con .NET 10

Arquitectura sencilla, código comprobado

Dapper + SQLite + ASP.NET Core + xUnit.net

Versión 2.0 · 6 de noviembre de 2026

Una guía práctica para construir aplicaciones mantenibles: responsabilidades claras, dependencias explícitas, reglas verificables y acceso a datos controlado.

El ejemplo conserva las categorías y los productos del planteamiento original. La presentación se adapta a una API HTTP con controladores para concentrar el aprendizaje en el diseño y las pruebas.

Repositorio y código completo

github.com/eliaquin/dotnet-development-guide

Base tecnológica verificada

.NET 10 LTS · SDK 10.0.403 · Runtime 10.0.14

La edición es una referencia fechada. Mantenga la rama LTS al día con los parches de seguridad y repita las comprobaciones antes de actualizar dependencias.

01 / Principios que se conservan

Un criterio común para mantener el software

La estandarización facilita entender y modificar el trabajo de otro desarrollador. La arquitectura debe hacer visibles las responsabilidades y permitir cambiar una implementación sin trasladar sus detalles a toda la solución.

- **Dominio:** expresa datos y reglas propias del catálogo, sin HTTP, SQL ni paquetes externos.
- **Aplicación:** coordina los casos de uso y depende de contratos de persistencia.
- **Infraestructura:** concentra Dapper, conexiones, SQL, transacciones y evolución del esquema.
- **Presentación:** recibe solicitudes, valida su forma y convierte resultados en respuestas HTTP.
- **Pruebas:** verifican reglas aisladas y el comportamiento completo con una base real.

Las dependencias se reciben por constructor. Los DTO de entrada controlan qué puede enviar el cliente; las respuestas se construyen explícitamente. Una escritura representa una operación terminada, sin separar públicamente “crear” de “guardar”.

Cómo recorrer la guía

Primero ejecute el repositorio (sección 03), pruebe el flujo HTTP (04) y después estudie las capas (05 a 10). Al añadir una funcionalidad, siga el proceso de la sección 13 y ejecute todos los controles de la sección 12.

Mapa de lectura: tecnología 02 · inicio 03 · API 04 · arquitectura 05 · reglas 06 · SQL 07 · Dapper 08 · concurrencia 09 · HTTP 10 · pruebas 11 · calidad 12 · evolución 13 · evidencia 14 · fuentes 15.

Este ejemplo es un catálogo local sin autenticación y sin interfaz gráfica. Una aplicación MVC con vistas puede reutilizar las mismas capas, conservando ViewModels y la validación de formularios.

02 / Tecnología y sustituciones

Usar lo necesario

.NET 10 es el LTS estable más reciente a la fecha de esta edición; la política oficial consultada indica soporte hasta el 14 de noviembre de 2028 [1]. Se utiliza C# 14, la versión predeterminada del SDK para net10.0, sin características preview.

Responsabilidad	Elección de esta edición
Plataforma	.NET 10, SDK 10.0.403 / runtime 10.0.14
Datos	Dapper 2.1.89 + Microsoft.Data.Sqlite 10.0.14
HTTP y OpenAPI	ASP.NET Core + Microsoft.AspNetCore.OpenApi 10.0.14
Pruebas	xunit.v3.mtp-v2 4.0.2 + Microsoft.Testing.Platform
Pruebas HTTP	Microsoft.AspNetCore.Mvc.Testing 10.0.14
DI, JSON y logs	Servicios nativos, System.Text.Json e ILogger

Entity Framework y su proveedor: se sustituyen por SQL explícito y Dapper. No quedan DbContext, DbSet, tracking, SaveChanges ni herramientas de migración de EF.

Autofac y su extensión: se retiran porque el contenedor integrado cubre las necesidades. **AutoMapper y su extensión:** se retiran porque los DTO pequeños pueden mapearse con claridad en código. Estas decisiones no significan que ambas bibliotecas estén obsoletas.

Identity con almacenes EF: se retira junto a EF. El catálogo local no implementa cuentas. Para una aplicación con usuarios, integre un proveedor de identidad mantenido y autorización por política; no traslade un almacén de Identity a Dapper sin un diseño específico.

xUnit.net v3 es una opción mantenida compatible con la plataforma de pruebas actual [8]. MSTest y NUnit también son opciones válidas; no existe una biblioteca universalmente obligatoria. El nombre del paquete continúa siendo xunit.v3.mtp-v2 aunque su versión publicada sea 4.0.2.

03 / Ejecutar desde cero

Requisitos y comprobación inicial

Instale el SDK .NET 10.0.403 desde el sitio oficial [2] y Git. Windows, Linux y macOS pueden ejecutar el ejemplo; no necesita SQL Server ni Docker. SQLite viene con el proveedor NuGet. Use una carpeta local con permiso de escritura.

```
git clone https://github.com/eliaquin/dotnet-development-guide.git
cd dotnet-development-guide
dotnet --version
dotnet restore --locked-mode
dotnet build Store.slnx --configuration Release --no-restore
dotnet test --solution Store.slnx --configuration Release --no-build
```

El SDK debe mostrar 10.0.403 o un parche posterior de la misma banda 10.0.4xx permitido por global.json. La restauración bloqueada detecta cambios inesperados frente a packages.lock.json. No ejecute pruebas con --no-build hasta completar la compilación.

Iniciar la API local

```
dotnet run --project src/Store.Api --configuration Release --no-build -- \
  --urls http://127.0.0.1:5080 --environment Development
```

El comando de inicio está dividido para Bash. En PowerShell ejecútelo en una sola línea. Use curl.exe para evitar el alias curl de algunas versiones de PowerShell.

```
curl -f http://127.0.0.1:5080/health
curl -f http://127.0.0.1:5080/openapi/v1.json
```

El primer inicio crea src/Store.Api/App_Data/store.db, aplica el esquema y deja el catálogo vacío. El siguiente conserva los datos. Database:Path admite otra ruta; Database__Path es su equivalente como variable de entorno. Las rutas relativas se resuelven desde el content root de la API. Detenga con Ctrl+C.

04 / Probar el caso completo

Crear, consultar y modificar

Estos comandos Bash suponen una base nueva. Copie los identificadores devueltos si ya tiene datos; no dependa del número 1 en una aplicación real. Los precios se expresan en DOP y admiten dos decimales. El ejemplo no convierte monedas.

```
curl -i http://127.0.0.1:5080/api/categories \
  -H 'Content-Type: application/json' -d '{"name":"Accesorios"}'

curl -i http://127.0.0.1:5080/api/gadgets \
  -H 'Content-Type: application/json' \
  -d '{"name":"Teclado","description":"USB","price":1499.90,"categoryId":1}'

curl -f 'http://127.0.0.1:5080/api/gadgets?categoryId=1&offset=0&limit=10'

curl -i -X PUT http://127.0.0.1:5080/api/gadgets/1 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Teclado","description":"USB","price":1399.90,"categoryId":1,"version":1}'

curl -i -X DELETE 'http://127.0.0.1:5080/api/gadgets/1?version=2'
```

Las creaciones devuelven 201 y Location. Una actualización válida devuelve el producto con la versión incrementada; el borrado devuelve 204. Repetir una actualización con una versión anterior devuelve 409 y conserva los datos actuales.

Recurso	Operaciones disponibles
/api/categories	GET con paginación; POST
/api/categories/{id}	GET
/api/gadgets	GET con filtro categoryId y paginación; POST
/api/gadgets/{id}	GET; PUT con version; DELETE con ?version=

Las categorías se crean y consultan; el CRUD completo se demuestra con productos. GET sin filtro lista todos los productos dentro de la página, ordenados por Id. offset empieza en 0; limit vale 50 por defecto y admite 1 a 100.

05 / Organización de la solución

Capas pequeñas y dependencias hacia el núcleo

```
src/
  Store.Domain/          Entidades y reglas puras
  Store.Application/     Casos de uso y contrato IStoreRepository
  Store.Infrastructure/  Dapper, SQLite y migración SQL embebida
  Store.Api/             Controladores, contratos y composición

tests/
  Store.UnitTests/        Dominio y servicios sin base de datos
  Store.IntegrationTests/ HTTP + SQLite temporal real

docs/                    Guía, decisiones y evidencia de validación
```

El flujo de ejecución es API -> StoreService -> IStoreRepository -> SqliteStoreRepository -> SQLite. La dirección de las referencias de proyectos es distinta: Application referencia Domain; Infrastructure referencia Application; Api referencia Application e Infrastructure para componer la aplicación.

Domain no depende de otro proyecto. Application desconoce Dapper y SQLite. Ningún controlador escribe SQL ni abre conexiones. La implementación de IStoreRepository se elige en Program.cs.

```
builder.Services.AddSingleton(TimeProvider.System);
builder.Services.AddSingleton(new SqliteConnectionFactory(databasePath));
builder.Services.AddSingleton<DatabaseInitializer>();
builder.Services.AddScoped<IStoreRepository, SqliteStoreRepository>();
builder.Services.AddScoped<StoreService>();
```

La fábrica singleton contiene una cadena de conexión, no una conexión compartida. Cada operación abre y desecha su propia conexión. El servicio y el repositorio viven dentro del scope de la solicitud.

No se agregan repositorios genéricos, mediadores ni una unidad de trabajo abstracta para este catálogo. Añada una abstracción cuando resuelva una necesidad concreta y comprobable; mantenga las operaciones de negocio identificables.

06 / Dominio y casos de uso

Validar antes de escribir

Category y Gadget son registros inmutables de resultados persistidos. GadgetDetails es el valor validado que necesita una escritura. CatalogRules centraliza el nombre: obligatorio, sin espacios externos y de hasta 50 caracteres. La descripción admite hasta 500 caracteres y se recorta.

El precio debe estar entre 0 y 1 000 000 DOP y tener como máximo dos decimales. Se guarda como un entero de centavos para evitar la aproximación binaria de REAL/double.

```
if (price < 0 || price > 1_000_000 || decimal.Round(price, 2) != price)
{
    throw new DomainRuleException(
        "El precio debe estar entre 0 y 1000000, con hasta dos decimales.");
}
```

La conversión final usa checked((long)(price * 100)). Un precio de 1499.90 se almacena como 149990 y vuelve al DTO mediante PriceCents / 100m. Todos los precios del catálogo pertenecen a la misma moneda.

El servicio coordina la operación

```
public Gadget CreateGadget(string name, string? description,
    decimal price, long categoryId)
{
    var details = GadgetDetails.Create(name, description, price, categoryId);
    _ = GetCategory(categoryId);
    return repository.AddGadget(details, clock.GetUtcNow());
}
```

La regla de precio se prueba sin HTTP ni base. El servicio comprueba la categoría y obtiene la hora desde TimeProvider, lo que permite pruebas deterministas. Los tiempos se generan en UTC y se serializan en formato de ida y vuelta.

El contrato del repositorio se declara en Application y devuelve datos o errores del caso de uso. DomainRuleException representa datos inválidos; MissingRecordException, ausencia; ConflictException, duplicados o versión obsoleta. Una excepción inesperada queda como error interno con trazabilidad.

07 / Esquema y evolución de datos

SQL legible y restricciones reales

La migración inicial está en src/Store.Infrastructure/Migrations/001_initial.sql y se incluye como recurso embebido. Las tablas STRICT evitan almacenar tipos incompatibles. El esquema refuerza las reglas del dominio con CHECK, UNIQUE y FOREIGN KEY.

```
CREATE TABLE Gadgets (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL
        CHECK (length(trim(Name)) BETWEEN 1 AND 50 AND Name = trim(Name)),
    Description TEXT NOT NULL CHECK (length(Description) <= 500),
    PriceCents INTEGER NOT NULL CHECK (PriceCents BETWEEN 0 AND 100000000),
    CategoryId INTEGER NOT NULL REFERENCES Categories(Id) ON DELETE RESTRICT,
    Version INTEGER NOT NULL DEFAULT 1 CHECK (Version > 0),
    CreatedUtc TEXT NOT NULL
) STRICT;
```

Categories tiene Id, Name y CreatedUtc. Su nombre es único con COLLATE NOCASE. La comparación NOCASE incorporada solo ignora diferencias de mayúsculas ASCII; no constituye normalización lingüística de nombres con acentos.

El índice IX_Gadgets_CategoryId_Id favorece el filtro por categoría con orden estable. AUTOINCREMENT evita reutilizar IDs borrados y confundir la versión de un producto nuevo con la de uno antiguo.

Inicio repetible y cambios controlados

DatabaseInitializer crea la carpeta, activa WAL y abre una transacción de escritura. Consulta PRAGMA user_version: aplica la migración cuando es 0, establece 1 y confirma; si ya es 1 no repite la creación. Rechaza bases con una versión superior a la que conoce.

Esta implementación contiene una migración inicial, no un motor general de migraciones. Para la versión 2, agregue un script nuevo, ejecútelo de forma secuencial y transaccional e incremente la versión. Pruebe actualización y rollback sobre una copia. No modifique silenciosamente el script 001 después de distribuirlo.

En una base compartida, separe la ejecución de migraciones del arranque normal, haga respaldo y coordine el despliegue. Nunca borre una base con datos para resolver una incompatibilidad del esquema.

08 / Dapper sin detalles ocultos

Consultas parametrizadas y conexiones breves

Dapper extiende la conexión ADO.NET con operaciones SQL y mapeo de resultados [3]. No genera el esquema ni rastrea cambios. La aplicación es responsable de definir consultas, transacciones y conversiones.

```
using var connection = connectionFactory.OpenConnection();
var row = connection.QuerySingleOrDefault<CategoryRow>(
    "SELECT Id, Name, CreatedUtc FROM Categories WHERE Id = @Id;",
    new { Id = id });
return row is null ? null : ToCategory(row);
```

Los parámetros transportan valores; los datos del usuario nunca se concatenan en SQL. Las interpolaciones presentes en el repositorio solo insertan una lista constante de columnas definida en el código. Use una lista permitida si alguna vez necesita variar un identificador SQL.

Los listados tienen columnas explícitas, ORDER BY Id, LIMIT y OFFSET. QuerySingleOrDefault representa ausencia sin NullReferenceException. Los modelos de fila usan tipos primitivos de SQLite y convierten CreatedUtc explícitamente con cultura invariante; se evita depender de conversiones implícitas de DateTimeOffset [4].

Particularidades de SQLite

La fábrica activa ForeignKeys=true, DefaultTimeout=5 y Pooling=false. Desactivar pooling simplifica el cierre y la limpieza de archivos del ejemplo. Mantenga la conexión dentro de using y la transacción en la misma conexión.

Microsoft.Data.Sqlite ejecuta las operaciones async de forma síncrona; Microsoft recomienda evitarlas [5]. Por ello el repositorio es síncrono y no utiliza Task.Run. El timeout limita la espera por bloqueos; no garantiza un límite total de duración ni la interrupción inmediata cuando el cliente cancela.

WAL mejora la convivencia de lectores y escritores, pero SQLite sigue admitiendo un escritor activo. Este diseño sirve para uso local y cargas moderadas. Para muchas escrituras concurrentes o varias instancias, evalúe un servidor SQL y el acceso asíncrono real del proveedor antes de ampliar el sistema.

09 / Transacciones y concurrencia

Una operación debe terminar completa

INSERT ... RETURNING obtiene el resultado de una escritura en la misma sentencia atómica. Para actualizar o borrar y distinguir errores, el repositorio abre una transacción de escritura y mantiene la comprobación dentro de ella.

```
UPDATE Gadgets
SET Name = @Name, Description = @Description, PriceCents = @PriceCents,
    CategoryId = @CategoryId, Version = Version + 1
WHERE Id = @Id AND Version = @ExpectedVersion
RETURNING Id, Name, Description, PriceCents, CategoryId, Version, CreatedUtc;
```

El cliente lee version=1 y la envía al modificar. Solo un escritor puede cambiar esa versión: si una segunda solicitud conserva la 1, no afecta filas. Si el Id existe, se devuelve 409; si ya no existe, 404. DELETE aplica la misma condición de versión.

Una edición con versión obsoleta no debe repetirse automáticamente con la versión nueva. El cliente debe consultar los cambios y resolver el conflicto. Las pruebas incluyen dos solicitudes simultáneas contra la misma versión.

Qué protege la base

La validación de categoría del servicio mejora el mensaje, pero no sustituye la FK: otro proceso puede cambiar los datos entre la lectura y la escritura. La restricción sigue protegiendo la integridad y su fallo conocido se traduce a ConflictException.

La unicidad de categorías se decide en SQLite, lo que evita la carrera de “consultar si existe y después insertar”. Se capturan únicamente los códigos específicos de duplicado y FK; los fallos inesperados no se convierten en un éxito ni se ocultan como datos inválidos.

Si un futuro caso de uso requiere varias escrituras, agregue una operación del repositorio que las ejecute en la misma conexión y transacción, pasando transaction a cada llamada Dapper. Confirme al final. Ante una excepción, deseche la transacción sin confirmar. No distribuya la transacción entre llamadas HTTP.

10 / Presentación y contrato HTTP

Controladores delgados y datos de entrada propios

[ApiController] y DataAnnotations validan las solicitudes antes de ejecutar la acción. Price, CategoryId y Version usan valores anulables con [Required] para distinguir un campo ausente de cero. Los DTO recortan los espacios antes de validar las longitudes. El dominio vuelve a comprobar las reglas cuando el servicio se invoca desde otro lugar.

```
public sealed class CreateCategoryRequest
{
    private string _name = string.Empty;

    [Required, StringLength(50)]
    public string Name
    {
        get => _name;
        init => _name = value?.Trim() ?? string.Empty;
    }
}
```

El mapeo de GadgetResponse es explícito; solo expone los campos acordados y convierte centavos a decimal. Las entradas no permiten al cliente establecer Id, CreatedUtc ni la nueva versión. System.Text.Json administra JSON sin agregar otro serializador.

Estado	Significado en este ejemplo
200	Lectura o modificación completada
201 + Location	Recurso creado y persistido
204	Producto eliminado
400	JSON, DTO, paginación o regla de negocio inválidos
404	Recurso o categoría requerida ausentes
409	Nombre duplicado o versión obsoleta
500	Fallo inesperado con detalle genérico

ApiExceptionHandler produce ProblemDetails para errores conocidos y evita devolver mensajes internos de excepciones inesperadas. El manejador registra explícitamente los fallos inesperados con ILogger y se agrega traceId para relacionar un reporte con su solicitud. No registre cuerpos completos ni secretos sin una necesidad concreta.

OpenAPI se publica en /openapi/v1.json únicamente en Development. No se añade una interfaz de documentación como dependencia. /health comprueba acceso real al catálogo a través del servicio.

11 / Estrategia de pruebas

Reglas aisladas y funcionamiento completo

Store.UnitTests usa xUnit.net v3, un repositorio fake pequeño y un TimeProvider controlado. Comprueba límites, valores inválidos, normalización, conversión de dinero, ausencia, versión y validación antes de escribir. No necesita conexión ni servidor.

Store.IntegrationTests utiliza WebApplicationFactory<Program> y un archivo SQLite temporal distinto por prueba. Ejecuta solicitudes HTTP reales dentro del host de pruebas: serialización, validación, servicios, Dapper, SQL y respuestas. Cada archivo se elimina al cerrar la prueba; no se usa la base del desarrollador.

```
{
  "sdk": {
    "version": "10.0.403",
    "rollForward": "latestPatch",
    "allowPrerelease": false
  },
  "test": { "runner": "Microsoft.Testing.Platform" }
}
```

Los proyectos de pruebas son ejecutables y habilitan UseMicrosoftTestingPlatformRunner. El SDK 10 ejecuta la plataforma indicada en global.json [8, 9]. No se requieren los paquetes del runner VSTest para este repositorio.

```
dotnet test --solution Store.slnx --configuration Release

dotnet test --project tests/Store.UnitTests/Store.UnitTests.csproj \
  --configuration Release

dotnet test --project tests/Store.IntegrationTests/Store.IntegrationTests.csproj \
  --configuration Release
```

Pruebe comportamientos: después de un conflicto los datos siguen iguales; un precio inválido no llega a persistencia; un texto que parece SQL se conserva como dato; el reinicio mantiene los registros. Evite pruebas que simplemente reproduzcan cada línea de la implementación.

Los escenarios de integración cubren además FK y restricciones SQL directas, idempotencia del esquema, health, OpenAPI y edición simultánea. La base usada en pruebas debe seguir siendo del mismo tipo que la del ejemplo.

12 / Controles de calidad repetibles

La calidad debe poder comprobarse

Directory.Build.props habilita nullable, analizadores recomendados, estilo en compilación, compilación determinista y TreatWarningsAsErrors. .editorconfig comparte el formato. Las versiones directas se centralizan en Directory.Packages.props y cada proyecto conserva packages.lock.json.

Ejecute estos controles desde la raíz antes de publicar un cambio:

```
dotnet restore --locked-mode
dotnet format Store.slnx --verify-no-changes --no-restore
dotnet build Store.slnx --configuration Release --no-restore
dotnet test --solution Store.slnx --configuration Release --no-build
dotnet list Store.slnx package --vulnerable --include-transitive
```

Bash puede ejecutar scripts/verify.sh, que agrupa la misma secuencia. Windows puede usar Git Bash o los comandos individuales. La auditoría también se realiza al restaurar; sus advertencias fallan la compilación por la política de warnings como errores.

GitHub Actions repite restauración bloqueada, formato, compilación, pruebas y auditoría en Ubuntu y Windows. Las acciones están fijadas a SHA y los permisos del workflow son de lectura. Mantenga los SHA actualizados y revise las notas de versión antes de cambiarlos.

Una actualización pequeña y revisable

1. Actualice el SDK/runtime dentro de .NET 10 LTS y las versiones pertinentes.
2. Restaure para regenerar los archivos lock de forma deliberada.
3. Revise el cambio de dependencias directas y transitivas, licencias y avisos.
4. Ejecute todos los controles y el flujo de inicio sobre una base nueva y otra existente.
5. Actualice la guía y la evidencia de validación con las versiones realmente probadas.

No suprima analizadores de forma general para conseguir una compilación verde. Use una excepción localizada y justificada solo cuando aporte claridad. Una ejecución satisfactoria acredita los escenarios incluidos, no una garantía absoluta de ausencia de defectos ni una prueba de carga.

13 / Añadir una funcionalidad

Un recorrido de implementación

Suponga que debe permitir desactivar un producto sin borrarlo. Antes de escribir un endpoint, defina el comportamiento: quién puede verlo, si puede modificarse y qué debe ocurrir con dos cambios simultáneos.

1. **Dominio:** agregue la regla y sus pruebas. Use un valor o estado explícito si evita combinaciones inválidas.
2. **Aplicación:** defina el caso de uso y el contrato mínimo de persistencia. Mantenga las decisiones de negocio fuera del controlador y del SQL.
3. **Datos:** cree la siguiente migración; conserve los datos anteriores. Implemente SQL parametrizado con condición de versión y restricciones apropiadas.
4. **Presentación:** agregue DTO y endpoint con respuestas documentadas. No permita que un mapeo automático habilite escritura de campos internos.
5. **Integración:** pruebe éxito, ausencia, entrada inválida y conflicto concurrente con SQLite real.
6. **Revisión:** ejecute los controles, revise el diff y actualice la documentación.

Conservar la sencillez

El servicio usa IStoreRepository porque marca una frontera real. No cree interfaces para cada clase ni duplique cada tabla en capas sin una necesidad. Mantenga nombres claros, una responsabilidad por tipo y métodos cortos que expresen una operación.

En proyectos mayores puede separar el repositorio por capacidades. Si una operación abarca varios repositorios, diseñe primero su frontera transaccional. No introduzca eventos, colas o microservicios para resolver una operación que cabe en una transacción local.

Para una presentación MVC, coloque ViewModels y validaciones HTTP en el proyecto web, compruebe ModelState, proteja formularios con antiforgery y aplique autorización cuando corresponda. El servicio y el repositorio no necesitan conocer las vistas.

Para migrar a un servidor SQL, sustituya el proveedor y las consultas de Infrastructure. Revise RETURNING, tipos, comparación de texto, migraciones y aislamiento. Cambiar únicamente la cadena de conexión no garantiza compatibilidad. Reejecute las pruebas de integración contra el proveedor elegido.

14 / Evidencia y alcance verificado

Resultados de esta edición

La evidencia detallada está en docs/validacion.md. El código, las versiones bloqueadas y los comandos de esta guía corresponden al mismo repositorio.

Comprobación	Resultado registrado
SDK / runtime	10.0.403 / 10.0.14
Compilación Release	Aprobada; cero advertencias y errores
Pruebas unitarias	55 aprobadas; cero omitidas
Pruebas de integración	47 aprobadas; cero omitidas
Formato y dependencias bloqueadas	Verificados
Auditoría NuGet directa y transitiva	Sin vulnerabilidades conocidas reportadas
Inicio local	Health, OpenAPI, CRUD y persistencia verificados

La verificación local se realizó en macOS ARM64. El workflow del repositorio permite repetirla en Ubuntu y Windows. Consulte su ejecución enlazada en la evidencia para conocer el estado remoto más reciente; no deduzca compatibilidad de una plataforma únicamente por su configuración.

Qué cubren las pruebas

- Reglas de nombres, descripción, importes, categoría y paginación.
- Hora UTC inyectable y validación antes de persistir.
- Creación, consulta, modificación y eliminación por HTTP.
- Duplicados, inexistencia, JSON inválido y campos obligatorios.
- SQL parametrizado, precisión monetaria y restricciones reales.
- Edición concurrente: solo una solicitud confirma la misma versión.
- Esquema repetible y persistencia después de reiniciar el host.

No se incluyen pruebas de carga, auditoría de seguridad externa ni un despliegue productivo. La autenticación y una interfaz gráfica quedan fuera del catálogo local. Estos límites permiten mantener el ejemplo ejecutable sin cuentas, servicios externos ni infraestructura adicional.

Para informar de un fallo, indique el commit, dotnet --info, el comando ejecutado, la respuesta y traceId si existe. Quite secretos y datos personales antes de compartir los logs.

15 / Fuentes y mantenimiento

Referencias primarias

Consultadas el 6 de noviembre de 2026. Las decisiones sobre librerías combinan estas fuentes con las necesidades del ejemplo; no implican que una sola arquitectura sirva para todos los proyectos.

[1] Política oficial de soporte .NET. LTS, parches y fecha de fin de soporte.

[2] Metadata oficial de .NET 10 y descarga de .NET 10. Versiones del SDK y runtime.

[3] Dapper: repositorio y documentación y paquete 2.1.89. Consultas, parámetros y transacciones.

[4] Dapper y Microsoft.Data.Sqlite. Conversión de tipos y particularidades del proveedor.

[5] Limitaciones async de SQLite y transacciones. Modelo de E/S y concurrencia.

[6] Inyección de dependencias en ASP.NET Core. Contenedor nativo y ciclos de vida.

[7] Validación MVC y OpenAPI. Contratos y documentación HTTP.

[8] xUnit.net v3 con Microsoft.Testing.Platform. Runner actual y proyectos ejecutables.

[9] dotnet test y Microsoft.Testing.Platform. Ejecución y selección del runner.

[10] Pruebas de integración ASP.NET Core. WebApplicationFactory y host de pruebas.

Código y documentación: repositorio público. Revise docs/decisiones.md para el razonamiento de las sustituciones y docs/validacion.md para los resultados reproducibles.